

CODECOMPASS EVALUATION

CS-Bookstore Bilingual QA Model Comparison

A controlled local-vs-cloud comparison over identical frozen evidence

Qwen: NOT_READY | GLM: READY_WITH_LIMITATIONS

Repository commit: 56ee148f3b4b8bc1f4eaaf921e22f280fa14ad7a
Questions: 12
Models: local Qwen and GLM
Status: frozen

Contents

1. Repository pin and index identity	7. Failure attribution
2. Controlled protocol	8. Citation analysis
3. Per-model aggregate results	9. Local-vs-cloud trade-off
4. Language and category breakdown	10. Limitations and verdicts
5. Per-question results	11. Reproducibility record
6. Exact captured answers	

1. Repository pin and index identity

The evaluated sample is a pinned multi-app Django repository. The frozen matrix covers model fields, method behavior, ORM relationships, request flow, form validation, and authentication logic across 9 target files.

Remote	https://github.com/AvaGhiasian/CS-Bookstore.git
Branch	main
Commit SHA	56ee148f3b4b8bc1f4eaaf921e22f280fa14ad7a
Commit date	2025-08-12T13:12:50+03:30
CodeCompass project	3 / CS-Bookstore

SOURCE FILES

56

SYMBOLS

144

CHUNKS

144

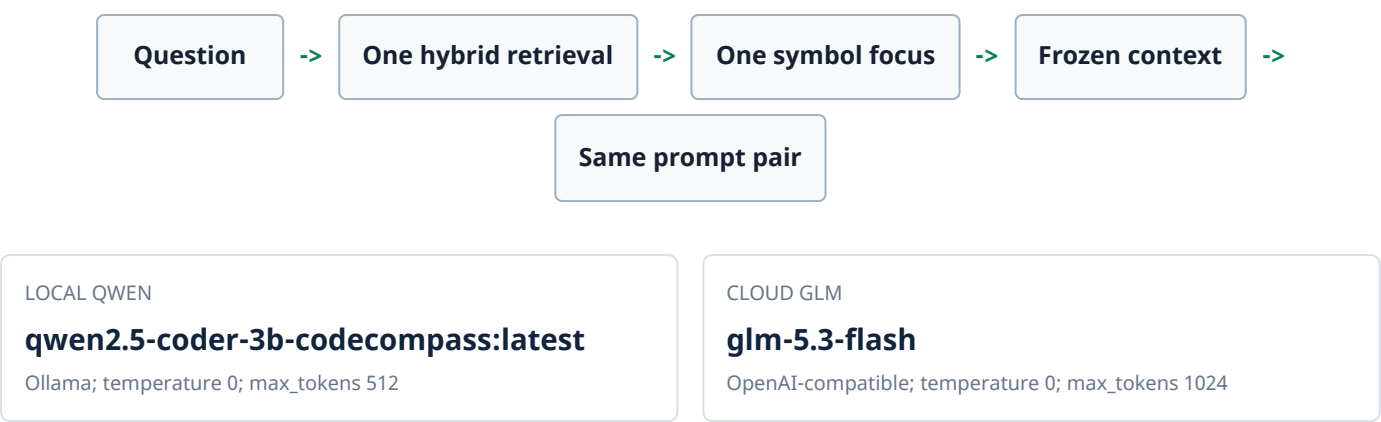
VECTORS

144

SQLite/Chroma IDs exact

Embedding: Ollama `nomic-embed-text-local:latest`, 768 dimensions. No re-indexing or embedding change occurred during the comparison.

2. Controlled protocol



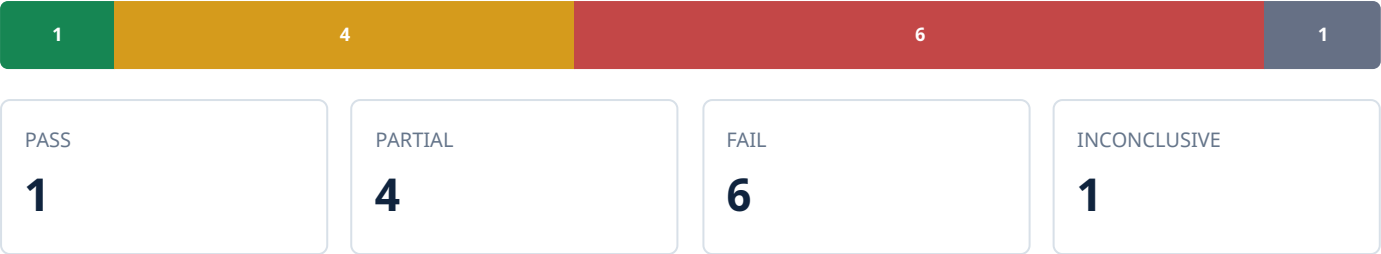
The exact context, system prompt, and user prompt hashes were shared between models for each question. Token budgets differ because GLM consumes completion budget for reasoning. This is not an equal-token benchmark.

Retrieval control: the expected target was retrieved and retained after focus for 12/12 questions. No retrieval failure was observed in this frozen sample.

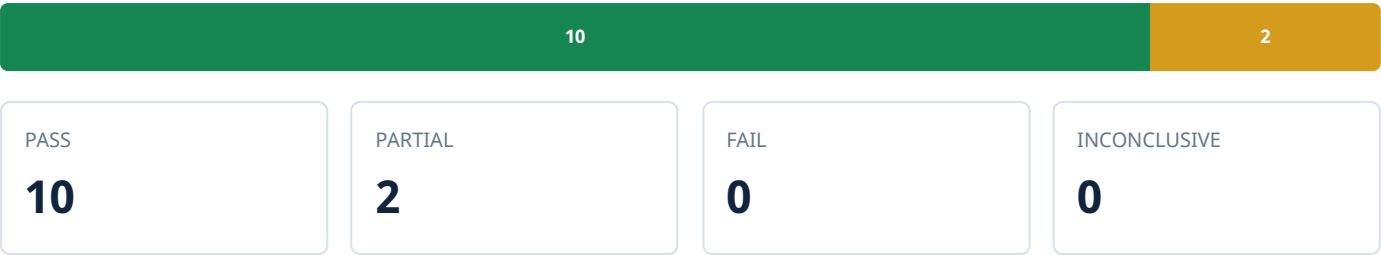
Matrix SHA-256	72c371f058a9301f034d291240f60545d3b3c85d5ae8fb9446a66fab24ee5252
Rubric SHA-256	f1a2af66613d1ee2713a1de6dc699603add0e5c4132c3b6875ae4d35e28234f7

3. Per-model aggregate results

Local Qwen



GLM



Metric	Local Qwen	GLM
Unsupported claims	7	0
Missing expected facts	24	3
Citation validity	14/14	14/14
Mean latency	24.277s	13.547s
Median latency	20.373s	11.717s
Verdict	NOT_READY	READY_WITH_LIMITATIONS

4. Language and category breakdown

Language	Local Qwen PASS	GLM PASS
English	1/6	6/6
Persian	0/6	4/6

Language adherence was 12/12 for Qwen and 12/12 for GLM. These counts do not support a general language-superiority claim.

Category	Qwen P/Pt/F/I	GLM P/Pt/F/I
authentication_user_logic	0/0/2/0	1/1/0/0
django_model_relationship	0/1/1/0	2/0/0/0
form_validation_behavior	0/1/1/0	2/0/0/0
method_behavior	0/1/1/0	2/0/0/0
model_field_structure	0/1/1/0	2/0/0/0
view_request_flow	1/0/0/1	1/1/0/0

5. Per-question results

ID	Lang	Target	Qwen	GLM	Latency Q/G
CSB-QA-01	EN	Book	FAIL	PASS	34.917s / 11.413s
CSB-QA-02	FA	User	PARTIAL	PASS	18.805s / 13.323s
CSB-QA-03	EN	Book.save	FAIL	PASS	7.948s / 11.004s
CSB-QA-04	FA	Post.delete	PARTIAL	PASS	29.942s / 12.510s
CSB-QA-05	EN	Review	FAIL	PASS	18.878s / 12.966s
CSB-QA-06	FA	Comment	PARTIAL	PASS	11.863s / 10.002s
CSB-QA-07	EN	review_book	PASS	PASS	21.867s / 12.021s
CSB-QA-08	FA	post_search	INCONCLUSIVE	PARTIAL	54.785s / 31.303s
CSB-QA-09	EN	UserRegisterForm.clean_password_2	PARTIAL	PASS	9.673s / 8.500s
CSB-QA-10	FA	CommentForm.clean_name	FAIL	PASS	27.505s / 9.100s
CSB-QA-11	EN	PhoneAuthBackend.authenticate	FAIL	PASS	17.006s / 9.856s
CSB-QA-12	FA	register	FAIL	PARTIAL	38.135s / 20.563s

BOTH_PASS	CSB-QA-07
GLM_ONLY_PASS	CSB-QA-01, CSB-QA-02, CSB-QA-03, CSB-QA-04, CSB-QA-05, CSB-QA-06, CSB-QA-09, CSB-QA-10, CSB-QA-11
QWEN_ONLY_PASS	None
BOTH_NON_PASS	CSB-QA-08, CSB-QA-12

6. Exact captured answers

Each answer is reproduced from the authoritative JSON. Both models received the same per-question context and prompt hashes.

CSB-QA-01

model_field_structure

Question: Which Book fields represent availability, inventory, base price, discount, and discounted price?

Target: store/models.py:40-76 / Book; target retained: True

context 7e25679497252599ec73350b29be6e9134e58bd3a4ee74a1e33fccc44b896e6d
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user c1c5d592b7408832997bab445eed820f6f45e7ec532017db41f2785655a3171f

Local Qwen FAIL 34.917s

The Book fields that represent availability, inventory, base price, discount, and discounted price are:

- `available`: A Boolean field indicating whether the book is available for purchase.
- `inventory`: A PositiveIntegerField representing the number of copies in stock.
- `price`: A PositiveIntegerField representing the base price of the book.
- `off`: A PositiveIntegerField representing the discount percentage applied to the book.
- `new\_price`: A PositiveIntegerField representing the discounted price of the book.

Missing: available is a BooleanField with default=True; inventory is a PositiveIntegerField with default=0; price is a PositiveIntegerField with default=0; off is a PositiveIntegerField with default=0; new_price is a PositiveIntegerField with default=0
Invented/unsupported: Claims that off represents a discount percentage, which is not explicit in the source.
Attribution: GENERATION

Fields and types were named, but all defaults were omitted and an unsupported percentage interpretation was added.

GLM PASS 11.413s

Based on the Book model in store/models.py (lines 40-76), the fields are:

Concept	Field	Definition
Availability	`available`	`models.BooleanField(default=True)`
Inventory	`inventory`	`models.PositiveIntegerField(default=0, verbose_name='موجودی')`
Base price	`price`	`models.PositiveIntegerField(default=0, verbose_name='قیمت')`
Discount	`off`	`models.PositiveIntegerField(default=0, verbose_name='تخفیف')`
Discounted price	`new_price`	`models.PositiveIntegerField(default=0, verbose_name='قیمت پس از تخفیف')`

All of the numeric fields (`inventory`, `price`, `off`, `new\_price`) are PositiveIntegerField with a default of 0, while `available` is a BooleanField defaulting to True.

Missing: None
Invented/unsupported: None
Attribution: NONE

All five fields, types, and defaults were reported without unsupported additions.

Trusted citations: file_id=85, symbol_id=607, chunk_id=9b28a4ad93db07e8f65e6f869a7973b48dcd5759e316d2eea35cf2324297049, store/models.py:40-76; file_id=85, symbol_id=608, chunk_id=f28d33b022186539b0040f427dcebe5c016d4a4bac0fdbf455ddf6990653d3d5, store/models.py:57-65

CSB-QA-02

model_field_structure

Question: مدل User چه فیلدهای اضافهای نسبت به AbstractUser تعریف می‌کند؟

Target: users/models.py:7-12 / User; target retained: True

```
context 3a122a1a2536375554c977a2437c7ebfade531f39cd5bc2b711aa48bbdea9d42
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 3c775dc815b54660fa121415ff877fe213f35caa534f176c12f8776502436f61
```

Local Qwen **PARTIAL** 18.805s

مدل User از فیلدهای زیر اضافهای تعریف می‌کند:

- `date\_of\_birth`: یک پروپرتی برای ذخیره تاریخ تولد کاربر
- `bio`: یک پروپرتی برای ذخیره بایو کاربر
- `photo`: یک پروپرتی برای ذخیره تصویر کاربر
- `job`: یک پروپرتی برای ذخیره شغل کاربر
- `phone`: یک پروپرتی برای ذخیره شماره تماس کاربر

Missing: date_of_birth is DateField(blank=True, null=True); bio is TextField(null=True, blank=True); photo is ImageField(upload_to="profile_images/", null=True, blank=True); job is CharField(max_length=25, null=True, blank=True); phone is CharField(max_length=11, null=True, blank=True)

Invented/unsupported: None**Attribution:** GENERATION

All field names were identified, but their required types and options were omitted.

GLM **PASS** 13.323s

بر اساس کد موجود در `users/models.py`، مدل `User` که از `AbstractUser` ارث‌بری می‌کند، **پنج فیلد اضافی** تعریف می‌کند:

1. `date_of_birth` — فیلد `DateField` با `blank=True, null=True` و نام توصیفی `verbose_name` «تاریخ تولد».
2. `bio` — فیلد `TextField` با `blank=True, null=True` و نام توصیفی `verbose_name` «بایو».
3. `photo` — فیلد `ImageField` با `blank=True, null=True` و نام توصیفی `verbose_name` «تصویر» و `upload_to="profile_images/"`.
4. `job` — فیلد `CharField` با `blank=True, null=True` و `max_length=25` و نام توصیفی `verbose_name` «شغل».
5. `phone` — فیلد `CharField` با `blank=True, null=True` و `max_length=11` و نام توصیفی `verbose_name` «شماره تماس».

همان‌طور که در کامنت کد نیز اشاره شده، فیلدهای `firstname`، `email`، `password`، `username` و `lastname` از قبل توسط `AbstractUser` تأمین شده‌اند و مدل `User` صرفاً فیلدهای فوق را به آنها اضافه می‌کند.

Missing: None**Invented/unsupported:** None**Attribution:** NONE

All five added fields and their declared options were reported.

Trusted citations: file_id=98, symbol_id=654, chunk_id=acddfafb34681cfe168ef2315a923275436f7eb181f1693cb7ba19340d81b686, users/models.py:7-12

CSB-QA-03

method_behavior

Question: How does Book.save populate the slug before saving?

Target: store/models.py:67-70 / Book.save; target retained: True

```
context 2fe87366a5894a7090e6722858863afc388170f5e6cda995c8e77c0da0b1e8c7
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 571bf00e1707736cb87249c32a28f9d714a6adcb7e9a86d41bacab9f49ce3ba3
```

Local Qwen FAIL 7.948s

Book.save populates the slug before saving by checking if the slug attribute is empty. If it is, it uses the slugify function from the slugify module to generate a slug based on the title attribute and assigns it to the slug attribute. This ensures that every book has a unique slug before being saved to the database.

Missing: it calls super().save(*args, **kwargs) afterwards
Invented/unsupported: Claims that the method ensures every book has a unique slug; uniqueness is not established by the source.
Attribution: GENERATION

The conditional slug generation was described, but the parent save call was omitted and uniqueness was invented.

GLM PASS 11.004s

According to the provided code (store/models.py, Book.save, lines 67–70), the slug is populated as follows:

- **Check for an existing slug**:** Before saving, the method checks if not self.slug: — i.e., it only generates a slug when the slug field is empty/falsy.
- **Generate the slug**:** If the slug is missing, it sets self.slug = slugify(self.title), meaning the slug is derived by applying slugify to the book's title.
- **Delegate to the parent save**:** After the slug is set (or left as-is if already present), it calls super().save(*args, **kwargs) to perform the actual save.

In short: Book.save auto-populates the slug from the title via slugify only when the slug is not already set, and then proceeds with the normal save.

Missing: None
Invented/unsupported: None
Attribution: NONE

The condition, non-replacement behavior, and parent save call were all reported.

Trusted citations: file_id=85, symbol_id=609, chunk_id=862dff324241a2bb0ceccd033852bd5c6a0f0b969162a4dbee8f077968772b17, store/models.py:67-70

CSB-QA-04 method_behavior

Question: متد Post.delete پيش از حذف خود پست با فايل‌هاى تصاويرش چه مى‌کند؟

Target: blog/models.py:74-78 / Post.delete; target retained: True

```
context 45772f18a4fd7062355aca03489962479c137f6a2bb3fb2b6b067c58f112a5c1
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 97b01b86692cb1950f44d599789829433dcc6634446a3e416e836e1df15ec6ec
```

Local Qwen PARTIAL 29.942s

متد `Post.delete` پيش از حذف خود پست با فايل‌هاى تصاويرش، بررسى مى‌کند که چگونه بايد فايل‌هاى تصاوير را حذف کنيد. اين متد به صورت زير عمل مى‌کند:

- با استفاده از `self.images.all()`، تمام تصاوير پست را دريافت مى‌کند.
- براى هر تصوير در ليست تصاوير، فايل آن را حذف مى‌کند.
- ابتدا، متد `storage.delete(path)` استفاده مى‌شود تا فايل را از سرويس ذخيره‌سازى (storage) حذف کند.

4. بعد از حذف فایل تصویر، متد پدر (`super().delete(*args, **kwargs)`) را استفاده می‌کند تا پست خود را به صورت عادی حذف کند.

این عملیات نشان می‌دهد که `Post.delete` از فایل‌های تصاویر پست برای حذف استفاده می‌کند و این فایل‌ها را از سرویس ذخیره‌سازی حذف می‌کند قبل از حذف پست خود.

Missing: for each image it obtains `img.image_file.storage` and `img.image_file.path`

Invented/unsupported: None

Attribution: GENERATION

The loop, storage deletion, and parent deletion were correct; the exact `image_file` storage/path extraction was omitted.

GLM PASS 12.510s

متد `Post.delete` (در `blog/models.py`، خطوط ۷۴-۷۸) پیش از حذف خود پست، فایل‌های تصاویر مرتبط با آن را از `storage` حذف می‌کند. به‌طور دقیق:

```
1. روی همه تصاویر مرتبط با پست پیمایش می‌کند: for img in self.images.all:
2. برای هر تصویر، storage و مسیر (path) فایل را از img.image_file استخراج می‌کند:
   python
   storage, path = img.image_file.storage, img.image_file.path
```

```
3. فایل هر تصویر را با فراخوانی storage.delete(path) از storage پاک می‌کند.
4. در نهایت با فراخوانی super().delete(*args, **kwargs) خود پست را حذف می‌کند.
```

به بیان خلاصه: این متد پیش از حذف رکورد پست، فایل فیزیکی تک‌تک تصاویر وابسته به آن را از `storage` حذف می‌کند تا فایل‌های یتیم باقی نمانند.

Missing: None

Invented/unsupported: None

Attribution: NONE

The complete cleanup sequence was reported.

Trusted citations: file_id=52, symbol_id=554, chunk_id=d34b861a3ab8320c85a5880e9c1f057a6d9cdb9575c2e6fefb09fbdcb5d8d738, `blog/models.py:74-78`

CSB-QA-05

django_model_relationship

Question: How is Review related to Book and the configured user model, and what deletion behavior is defined?

Target: `store/models.py:95-113 / Review`; target retained: True

```
context 7632f8cd5ca3f31934d72cb8815f45a40ca7b2978ccc1c0c9c56de10e2b2e0dc
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 07bcd5f45d96f5a310600f30407c1cefab4905bfcd5565391acbddefdd927681
```

Local Qwen FAIL 18.878s

Review is related to Book and the configured user model through foreign key relationships. Specifically:

- `book`: This field is a `ForeignKey` that references the `Book` model. It establishes a one-to-many relationship where each Review can be associated with exactly one Book.
- `user`: This field is also a `ForeignKey` that references the configured user model (likely `User` from Django's built-in authentication system). It establishes a one-to-many relationship where each Review can be associated with exactly one User.

The deletion behavior for both foreign key relationships is defined as CASCADE. This means that if a Book or User is deleted, all Reviews associated with it will also be deleted.

Missing: book is a ForeignKey to Book with related_name="reviews" and on_delete=models.CASCADE

Invented/unsupported: Speculates that the configured user model is likely Django's built-in User; the source only establishes settings.AUTH_USER_MODEL and this repository uses a custom user model.

Attribution: GENERATION

Both foreign keys and cascade behavior were described, but related_name=reviews was omitted and the user-model identity was speculated about.

GLM PASS 12.966s

Based on the provided code (store/models.py, Review class):

****Relationship to Book:****

- Review has a book field defined as models.ForeignKey(Book, ...).
- The reverse relation name is set via related_name="reviews", so reviews can be accessed from a Book instance (e.g., book.reviews).
- The field has the verbose name "کتاب".

****Relationship to the configured user model:****

- Review has a user field defined as models.ForeignKey(settings.AUTH_USER_MODEL, ...), meaning it points to the user model configured in Django settings rather than a hardcoded user class.
- The field has the verbose name "کاربر".

****Deletion behavior:****

- Both foreign keys use on_delete=models.CASCADE.
- Deleting a Book will cascade-delete its related Review records.
- Deleting a user (from the configured AUTH_USER_MODEL) will cascade-delete that user's Review records.

Other fields on Review visible in the context (not part of the relationships): rating (IntegerField with choices 1-5), body (TextField), created (DateTimeField with auto_now_add=True), updated (DateTimeField with auto_now=True), and active (BooleanField, default False).

Missing: None

Invented/unsupported: None

Attribution: NONE

Both relationships, related_name, and cascade behavior were reported.

Trusted citations: file_id=85, symbol_id=614, chunk_id=2fc2db0ab0373e4cc60c17372a02d577b919345806b2bb094487efc2cf1b2110, store/models.py:95-113

CSB-QA-06

django_model_relationship

Question: مدل Comment چگونه به Post متصل است و نام دسترسی معکوس این رابطه چیست؟

Target: blog/models.py:81-98 / Comment; target retained: True

context 6c8216b1aee1f4b16718e679f5940b49f890d4ec7e265bf30ef625da522143ba
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 170f9ea8381fdcb7d3b5b82dc9a6a2b94983f725fb1349f50ab08464ea6597c1

Local Qwen **PARTIAL** 11.863s

مدل `Comment` به `Post` متصل است و نام دسترسی معکوس این رابطه "comments" است.

Missing: post is a ForeignKey to Post; the relationship uses on_delete=models.CASCADE

Invented/unsupported: None

Attribution: GENERATION

The reverse name was correct, but ForeignKey and CASCADE were omitted.

GLM **PASS** 10.002s

مدل `Comment` از طریق یک فیلد `ForeignKey` به نام `post` به مدل `Post` متصل است. این رابطه با تنظیمات زیر تعریف شده است:

با حذف پست، کامنت‌های مرتبط با آن نیز حذف می‌شوند: `**on_delete=models.CASCADE**`
نام دسترسی معکوس این رابطه است: `**related_name="comments"**`

نام دسترسی معکوس این رابطه `**comments**` است؛ یعنی از روی یک نمونه از `Post` می‌توان به کامنت‌های آن دسترسی داشت (مثلاً `.post.comments.all()`)

کد مربوطه (فایل `blog/models.py`، کلاس `Comment`):

```
python
post = models.ForeignKey(Post, on_delete=models.CASCADE, related_name="comments", verbose_name="پست")
```

Missing: None

Invented/unsupported: None

Attribution: NONE

The relationship type, deletion behavior, and reverse name were all reported.

Trusted citations: file_id=52, symbol_id=555, chunk_id=6ff162dfdb7a7d2a59fd9de37cc8911cf0e9447a314a1e2b27d7a82fae9e72b4, blog/models.py:81-98; file_id=52, symbol_id=557, chunk_id=84bc0d4d5d9d47ff4fd592e34ff5651e8241d11bbe312a3cb59a3abb21ffd465, blog/models.py:97-98

CSB-QA-07

view_request_flow

Question: What happens in review_book when a valid POST review is submitted?

Target: store/views.py:95-107 / review_book; target retained: True

context 2f4b54f8ad1c1aec7289f5d010775de63dd17048518ea1247fc35b2cea29101e
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 2f4f9673ad9a1a4e614331adc1c8e1b3612ba41dbe70b6b0ac77e381b2447958

Local Qwen **PASS** 21.867s

When a valid POST review is submitted in the `review_book` function, the following happens:

1. The function retrieves a book object using `get_object_or_404(Book, id=pk, slug=slug)`.
2. It checks if the request method is 'POST'.
3. If it is, it creates an instance of `ReviewForm` with the POST data.
4. It verifies if the form is valid using `review_form.is_valid()`.
5. If the form is valid, it creates a new `Review` object by calling `review_form.save(commit=False)`, which prevents saving to the database immediately.
6. It assigns the book and user attributes of the review object to the corresponding values from the request (book ID and

current user).

7. Finally, it saves the review object to the database using `review.save()`.

8. After saving, it redirects the user to the absolute URL of the book using `redirect(book.get_absolute_url())`.

Missing: None

Invented/unsupported: None

Attribution: NONE

The complete valid-POST flow was reported.

GLM PASS 12.021s

When a valid POST review is submitted in `review_book` (store/views.py, lines 95–107), the following happens:

1. ****Book lookup****: The view first retrieves the book with `get_object_or_404(Book, id=pk, slug=slug)`.
2. ****Form validation****: Since the request method is 'POST', a `ReviewForm` is instantiated with `data=request.POST`, and `review_form.is_valid()` is checked.
3. ****If the form is valid****:
 - `review = review_form.save(commit=False)` — a `Review` instance is created from the form data but ****not yet saved**** to the database (due to `commit=False`).
 - `review.book = book` — the review is associated with the retrieved book.
 - `review.user = request.user` — the review is associated with the currently logged-in user.
 - `review.save()` — the review is now saved to the database.
 - `return redirect(book.get_absolute_url())` — the user is redirected to the book's own URL (its detail page).

Note that the redirect to `book.get_absolute_url()` also occurs if the request is not a POST or if the form is invalid, since that is the fall-through return at the end of the view.

Missing: None

Invented/unsupported: None

Attribution: NONE

The complete valid-POST flow was reported; the additional fall-through note was also source-grounded.

Trusted citations: file_id=88, symbol_id=625, chunk_id=b049cd3fc50eaf7207561f81dbe7c88abf916d7732b5f8831b41ac908f055ad4, store/views.py:95-107

CSB-QA-08

view_request_flow

Question: نمای `post_search` پس از دریافت `query` معتبر چگونه نتایج جست‌وجو را می‌سازد؟

Target: `blog/views.py:77-94 / post_search`; target retained: True

context 30d6b857f408c682d626e8b0d7cbad1c907776177e3229f6be8a5793386a7950
 system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
 user eaa9c19b1d27cb84d14963a7d2f86abaf411e573aeedec93564260f685d05b04

Local Qwen INCONCLUSIVE 54.785s

پس از دریافت یک پرسش معتبر، نمای `post_search` نتایج جست‌وجو را با استفاده از دو قسمت مختلف انجام می‌دهد:

1. ****قسمت اول****:

محاسبه می‌شود که شامل پست‌ها است که در عنوان آن‌ها تشابه با پرسش مورد نظر دارد `'results_1'` -

- این قسمت با استفاده از `'TrigramSimilarity'` و فیلتر برای پست‌های منتشر شده، پست‌ها را جست‌وجو می‌کند که تشابه بزرگتر 10% دارند.

- نتایج این قسمت با استفاده از `order\_by('-similarity')` مرتب می‌شوند بر اساس تشابه بزرگتر.

2. ****قسمت دوم:****

محاسبه می‌شود که شامل پست‌ها است که در توضیحات آن‌ها تشابه با پرسش مورد نظر دارد `results\_2` -
 این قسمت همچون قسمت اول، با استفاده از `TrigramSimilarity` و فیلتر برای پست‌های منتشر شده، پست‌ها را جست‌وجو می‌کند که تشابه بزرگتر 10% دارند.

- نتایج این قسمت همچون قسمت اول، با استفاده از `order\_by('-similarity')` مرتب می‌شوند بر اساس تشابه بزرگتر.

3. ****جمع نتایج:****

- نتایج از دو قسمت بالا با استفاده از `|` (union) جمع می‌شوند.
 - نتایج جمع شده با استفاده از `order\_by('-similarity')` مرتب می‌شوند بر اساس تشابه بزرگتر.

در نتیجه، نمای `post_search`

Missing: when query is present in request.GET, it validates SearchForm(data=request.GET); it reads query from form.cleaned_data; it renders blog/search.html with query and results

Invented/unsupported: None

Attribution: INCONCLUSIVE

The output visibly ends mid-sentence. The adapter does not retain finish_reason, so no retry or token-limit claim was made; the result is conservatively inconclusive.

GLM PARTIAL 31.303s

بر اساس کد `post\_search` در `blog/views.py` (خطوط ۷۷-۹۴)، پس از معتبر بودن فرم، ساخت نتایج به این شکل انجام می‌شود:

1. ****استخراج عبارت جست‌وجو:**** مقدار پاک‌سازی‌شده از فرم گرفته می‌شود:

```
python
query = form.cleaned_data['query']
```

2. ****ساخت دو کوئری‌ست با شباهت سه‌گانه (Trigram):****

TrigramSimilarity('title', `similarity`)، یک فیلد محاسباتی به نام `Post.published` روی پست‌های منتشرشده (:`results\_1` -
 مرتب می‌شود (شباهت بیشتر اول) `order\_by('-similarity')` فیلتر و با `similarity\_\_gte=0.1` اضافه می‌شود و سپس با `query`
 انجام می‌شود `description` همین کار با فیلد :`results\_2` -

```
python
Post.published.annotate(similarity=TrigramSimilarity('description', query)).filter(
    similarity__gte=0.1).order_by('-similarity')
```

3. ****ترکیب دو کوئری‌ست:**** دو نتیجه با عملگر `|` ترکیب می‌شوند و دوباره بر اساس شباهت به‌صورت نزولی مرتب می‌شوند:

```
python
results = (results_1 | results_2).order_by('-similarity')
```

4. ****ارسال به قالب:**** در نهایت `query` و `results` در دیکشنری `context` قرار می‌گیرند و قالب `blog/search.html` رندر می‌شود:

```
python
context = {
    'query': query,
    'results': results,
}
return render(request, "blog/search.html", context)
```

به‌طور خلاصه: جست‌وجو روی دو فیلد `title` و `description` پست‌های منتشرشده با آستانه شباهت `0.1` انجام می‌شود، نتایج هر دو فیلد با هم ترکیب و بر اساس میزان شباهت نزولی مرتب شده و همراه با عبارت جست‌وجو به قالب `blog/search.html` ارسال می‌شوند.

Missing: when query is present in request.GET, it validates SearchForm(data=request.GET)
Invented/unsupported: None
Attribution: GENERATION

Search construction and rendering were correct, but the request.GET condition and SearchForm(data=request.GET) validation step were not stated.

Trusted citations: file_id=57, symbol_id=571, chunk_id=e6e4fe2c5da879ce02ecbb1f5643604cc6e3fdfce3e84802c15c95a7f019d2bf, blog/views.py:77-94

CSB-QA-09

form_validation_behavior

Question: How does UserRegisterForm.clean_password_2 validate the repeated password?

Target: users/forms.py:22-25 / UserRegisterForm.clean_password_2; target retained: True

context 8a891e83afdc9722e3f4272beece21ed53ff7dd017e31001efe9f88fbae80fb6
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user bf10ba9ebd20eb5d5eb48f24f335549fc5085792b623abd6aa4c2b7351862fce

Local Qwen **PARTIAL** 9.673s

UserRegisterForm.clean_password_2 validates the repeated password by comparing it to the first password field. It checks if self.cleaned_data['password'] is equal to self.cleaned_data['password_2']. If they are not equal, it raises a ValidationError with the message 'پسوردها مطابقت ندارند'.

Missing: when they match, it returns cleaned_data["password_2"]
Invented/unsupported: None
Attribution: GENERATION

The comparison and mismatch error were correct, but the successful return of password_2 was omitted.

GLM **PASS** 8.500s

UserRegisterForm.clean_password_2 (in users/forms.py, lines 22-25) validates the repeated password as follows:

1. It compares the two cleaned field values: self.cleaned_data['password'] (the original password) against self.cleaned_data['password_2'] (the repeated password).
2. If the two values are not equal, it raises forms.ValidationError with the message 'پسوردها مطابقت ندارند' ("The passwords do not match!").
3. If they match, it returns self.cleaned_data['password_2'], confirming the repeated password is valid.

In short, the method enforces that the confirmation field password_2 exactly matches the password field, raising a validation error otherwise.

Missing: None
Invented/unsupported: None
Attribution: NONE

Both branches and the exact validation message were reported.

Trusted citations: file_id=93, symbol_id=637, chunk_id=e4ac744253be681fa0da265ef89cadbe5cd2e3ec6525bf2005d7886fb803d1d6, users/forms.py:22-25

CSB-QA-10

form_validation_behavior

Question: متد CommentForm.clean_name چه نام‌هایی را رد می‌کند و در حالت معتبر چه برمی‌گرداند؟

Target: blog/forms.py:45-51 / CommentForm.clean_name; target retained: True

context 54f18e9f5ef0f1795038ec5fb48a85e184227dbaaa72f8a67c36ea587d8961c8
 system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
 user 6c81dfe56b401f35a72c93f3c467a7f95256aa86fd2dccb88448ce97b488daa7

Local Qwen **FAIL** 27.505s

متد `CommentForm.clean\_name` رد می‌کند:

1. "نام معتبر نیست!" را در حالت معتبر برمی‌گرداند.

2. این متد یک فیلد نام (`name`) را بررسی می‌کند و از آن استفاده می‌کند.

3. اگر `name` خالی نباشد، فیلد را بررسی می‌کند:

- اگر طول `name` بین 2 تا 250 کاراکتر باشد، فیلد را به عنوان نام معتبر درخواست می‌دهد.

- اگر طول `name` کمتر از 2 یا بیشتر از 250 کاراکتر باشد، یک خطا (`ValidationError`) را به عنوان "نام معتبر نیست!" بررسی می‌دهد.

4. در حالت معتبر، `name` را به عنوان نام معتبر برمی‌گرداند.

Missing: a falsy name reaches the implicit None return**Invented/unsupported:** Contradictorily says that the valid case returns the error message 'نام معتبر نیست'.**Attribution:** GENERATION

Later statements describe the length checks correctly, but the opening contradiction is material and the falsy fall-through was omitted.

GLM **PASS** 9.100s

بر اساس کد `CommentForm.clean\_name` در فایل `blog/forms.py` (خطوط ۴۵ تا ۵۱):

****نام‌هایی که رد می‌شود:****- نام‌هایی که طولشان ****بیشتر از ۲۵۰ کاراکتر**** است (`len(name) > 250`)- نام‌هایی که طولشان ****کمتر از ۲ کاراکتر**** است (`len(name) < 2`)

در این دو حالت، متد خطای `forms.ValidationError` با پیام "نام معتبر نیست!" ایجاد (raise) می‌کند.

****در حالت معتبر:****

اگر طول نام بین ۲ تا ۲۵۰ کاراکتر باشد، متد همان `name` را برمی‌گرداند (`return name`).

****نکته اضافی (قابل مشاهده در کد):**** کل بررسی داخل شرط `if name` قرار دارد؛ بنابراین اگر `name` خالی/falsy باشد، هیچ‌کدام از

بررسی‌های طول انجام نمی‌شود و متد بدون مقدار بازگشتی صریح (یعنی عملاً `None` پایان می‌یابد).

Missing: None**Invented/unsupported:** None**Attribution:** NONE

Both length boundaries, valid return, and falsy fall-through were reported.

Trusted citations: file_id=49, symbol_id=536, chunk_id=82ed47caefdd51b242f09859d1d8f4b25e6b7e2dfd40c032212672896f024738, blog/forms.py:45-51

CSB-QA-11

authentication_user_logic

Question: How does PhoneAuthBackend.authenticate handle valid credentials, a wrong password, and missing or duplicate phone matches?

Target: users/authentication.py:5-13 / PhoneAuthBackend.authenticate; target retained: True

```
context 08a4772b574b750d511d7b9a10ca536fbdecac45333cbc06b377d4fc74a1f1fc
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 5fe2306e254a983d1178d636f3424b44b0a7e27e0e77432a80ae3e2efe16e665
```

Local Qwen **FAIL** 17.006s

PhoneAuthBackend.authenticate handles valid credentials by checking if the provided username matches a phone number in the database and then verifying the password using the check_password method. If both checks pass, it returns the user object.

For a wrong password, the function will catch the User.DoesNotExist or User.MultipleObjectsReturned exception when trying to retrieve the user by phone number. In this case, it will return None.

If there are missing or duplicate phone matches, the function will also catch these exceptions and return None. This is because the get method in Django raises a DoesNotExist exception if no object is found, and a MultipleObjectsReturned exception if multiple objects match the query.

Missing: None

Invented/unsupported: Incorrectly explains a wrong password as causing DoesNotExist or MultipleObjectsReturned during lookup; the source returns None after check_password is false.

Attribution: GENERATION

The final outcomes were listed, but the wrong-password control flow was materially misexplained.

GLM **PASS** 9.856s

Based on the code in users/authentication.py (lines 5–13), PhoneAuthBackend.authenticate works as follows:

****Valid credentials****

- The method treats the username parameter as a phone number and looks up the user with User.objects.get(phone=username).
- If a user is found and user.check_password(password) returns True, the user object is returned.

****Wrong password****

- If the user is found but user.check_password(password) returns False, the method returns None (no exception is raised; it simply falls through to return None).

****Missing phone match****

- If no user exists with the given phone number, User.DoesNotExist is raised by User.objects.get(...). The method catches this exception and returns None.

****Duplicate phone matches****

- If more than one user matches the phone number, User.MultipleObjectsReturned is raised by User.objects.get(...). This is also caught by the same except clause, and the method returns None.

In summary: the method returns the User only when exactly one user matches the phone and the password check succeeds; in every other case (wrong password, no match, or multiple matches), it returns None.

Missing: None

Invented/unsupported: None

Attribution: NONE

All lookup, password, and exception branches were correctly distinguished.

Trusted citations: file_id=92, symbol_id=629, chunk_id=b85cf00bb8e70be4b59b12060a2f4391f158513ec2026487a6b1f992560a3ed6, users/authentication.py:5-13

CSB-QA-12

authentication_user_logic

Question: تابع register رمز عبور کاربر جدید را چگونه ذخیره می‌کند و چه زمانی صفحه موفقیت را نمایش می‌دهد؟

Target: users/views.py:35-45 / register; target retained: True

```
context 6234c2aa8be94eeddfa5e17b68c60a520487fd9c912d33fc937c20a43633391e
system 8e7582633aeeb7ee199d61a7cd54e8238de1f91154e54ad6927522f49196407c
user 009e615abbde3d3184fcad961abeb13bc9e25a8afe70b0cb8c8a8ee1a7ce1442
```

Local Qwen FAIL 38.135s

تابع `register` در کد context ذخیره رمز عبور کاربر جدید به این صورت می‌کند:

- ابتدا، فرم `UserRegisterForm` با داده‌های پست شده توسط کاربر ساخته می‌شود.
- بررسی از اعتبارات فرم استفاده می‌شود.
- اگر فرم اعتبارسنجی شده باشد، کاربر جدید ساخته می‌شود و رمز عبور آن به صورت پیشفرض ذخیره می‌شود (با استفاده از `set\_password`).
- سپس کاربر ذخیره شده با فرم نمایش داده می‌شود.

صفحه موفقیت نمایش داده می‌شود:

- در صورت اعتبارسنجی و ذخیره کردن کاربر جدید، از `render` استفاده شده تا فرم نمایش داده شود.
- فرم نمایش داده شده شامل پیام موفقیت و اطلاعات کاربر ذخیره شده است.

پس، صفحه موفقیت نمایش داده می‌شود چنین زمانی که کاربر جدید با رمز عبور صحیح ثبت نام می‌کند.

Missing: when valid, it calls form.save(commit=False); after a valid POST it renders users/registration/register_done.html with the user; on GET it creates a blank UserRegisterForm and renders users/registration/register.html; an invalid POST renders users/registration/register.html with the bound form

Invented/unsupported: Describes the success response as displaying a form with a success message, which is not shown by the source.; Says success occurs when the user registers with a correct password rather than when the full form is valid.

Attribution: GENERATION

set_password and saving were mentioned, but commit=False and both non-success branches were omitted and the success rendering was mischaracterized.

GLM PARTIAL 20.563s

بر اساس کد موجود در `users/views.py` (تابع `register`، خطوط ۳۵-۴۵):

****نحوه ذخیره رمز عبور:****

- ابتدا با `form.save(commit=False)` یک شیء کاربر ساخته می‌شود اما هنوز در دیتابیس ذخیره نمی‌شود.
- سپس با فراخوانی `user.set\_password(form.cleaned\_data['password'])` رمز عبور پاک‌سازی‌شده (cleaned data) از فرم روی شیء کاربر تنظیم می‌شود.
- در نهایت با `user.save()` کاربر (همراه با رمز عبور تنظیم‌شده) در دیتابیس ذخیره می‌شود.

****زمان نمایش صفحه موفقیت:****

صفحه موفقیت (`users/registration/register\_done.html`) فقط زمانی رندر و بازگردانده می‌شود که هر دو شرط زیر برقرار باشند:

- درخواست از نوع `POST` باشد، و
- فرم `UserRegisterForm` معتبر باشد (`form.is\_valid`) مقدار True برگرداند).

در این حالت، پس از ذخیره کاربر، صفحه موفقیت با کانتکست `{ 'user': user }` نمایش داده می‌شود. در غیر این صورت (درخواست GET یا فرم نامعتبر)، صفحه `users/registration/register.html` همراه با فرم نمایش داده می‌شود

Missing: on POST it constructs UserRegisterForm(request.POST); on GET it creates a blank UserRegisterForm and renders users/registration/register.html

Invented/unsupported: None

Attribution: GENERATION

Password handling and success conditions were correct; explicit POST form construction and creation of a blank GET form were omitted.

Trusted citations: file_id=101, symbol_id=657, chunk_id=6bce017ec5405b5a1b9875b630a02a9ec8fee65df72f2d41e2586b77a16492a8, users/views.py:35-45

7. Failure attribution

RETRIEVAL FAILURES

0

shared for both models

TARGETS RETAINED

12/12

All expected target chunks were present after retrieval and focus. Qwen had 10 generation-attributed non-PASS results plus one visibly truncated/inconclusive answer. GLM had 2 generation-attributed omissions. No provider failure was recorded.

The Qwen response to [CSB-QA-08](#) ended mid-sentence. The provider abstraction did not retain `finish_reason`; the artifact therefore records the result conservatively as INCONCLUSIVE rather than asserting a token-limit cause.

8. Citation analysis

QWEN CITATIONS

14/14

GLM CITATIONS

14/14

Fourteen citation records per model were hydrated from trusted SQLite metadata. All mapped to the indexed project, file, chunk, symbol, relative path, and line range. Generation errors did not alter citation identity. Two questions retained a second same-root context block; those records remained valid but represent minor citation/context noise.

9. Local-vs-cloud trade-off

Dimension	Local Qwen	GLM
Observed answer quality	1/12 PASS under the frozen strict rubric	10/12 PASS; two omission-only PARTIAL results
Execution	Offline/local	Cloud/OpenAI-compatible
Privacy	Code context remains local during generation	Prompt/context sent to configured cloud endpoint
Generation cost	No cloud-generation charge	Usage/cost not retained in artifact
Observed latency	24.277s mean	13.547s mean

GLM produced more complete grounded answers in this controlled matrix. This does not establish universal model superiority.

10. Limitations and model verdicts

- Small frozen sample of 12 questions from one pinned repository.
- Single generation per model/question; no variance estimate or statistical significance claim.
- Model-specific token budgets prevent artificial truncation but do not form an equal-token benchmark.
- Provider usage, finish_reason, reasoning tokens, and cost are not retained by the current provider response model.

Local Qwen: NOT_READY

Not ready for this strict QA workload in the tested configuration. Offline and privacy advantages remain relevant.

GLM: READY_WITH_LIMITATIONS

Strong result in this frozen sample, with two observed omission-only limitations.

11. Reproducibility record

Experiment ID	cs_bookstore_bilingual_qa_v1
Matrix ID	cs_bookstore_bilingual_qa_matrix_v1
Matrix SHA-256	72c371f058a9301f034d291240f60545d3b3c85d5ae8fb9446a66fab24ee5252
Rubric SHA-256	f1a2af66613d1ee2713a1de6dc699603add0e5c4132c3b6875ae4d35e28234f7
Repository commit	56ee148f3b4b8bc1f4eaa921e22f280fa14ad7a
Executions	12 Qwen + 12 GLM; retries 0
Retrieval	Hybrid; one retrieval/focus/context/prompt build per question
Embedding	Ollama / <code>nomic-embed-text-local:latest</code> / 768 dimensions

No usage, finish_reason, reasoning-token, or estimated-cost values were invented. No new generation, retrieval, or benchmark execution was performed for this PDF.